

Bus / Transport Booking System

A Command-Line Python Application

Project Report

Submitted for the course: **Python Essentials**

Priyank Ramesh

26MEI1007

1. Introduction

Public transport systems rely heavily on organized booking and record keeping to manage passengers, seats, and revenue. This project is a simplified simulation of a bus booking counter, built entirely in Python and run from the command line. It was created as part of the Python Essentials course to apply core programming concepts — functions, data structures, file handling, and control flow — to a realistic, relatable problem.

2. Problem Statement

Manually tracking bus routes, seat availability, and passenger bookings on paper or in spreadsheets is slow and error-prone, especially as the number of routes and passengers grows. The goal of this project is to build a lightweight, terminal-based application that lets an operator view routes, book and cancel seats, look up a passenger's bookings, and calculate fares, while keeping all records safely stored between sessions.

3. Objectives

- Design a simple, readable data model to represent bus routes and bookings.
- Implement core booking operations: view routes, book a seat, cancel a booking.
- Allow searching of a passenger's booking history by name.
- Provide a fare calculator that works independently of the booking process.
- Persist all data between program runs using a local JSON file.
- Keep the entire application runnable from the command line with no external dependencies.

4. Technology Used

Component	Choice	Reason
Language	Python 3.7+	Course language; simple syntax for rapid development
Data storage	JSON (via built-in json module)	Human-readable, no database setup needed
Interface	Command-line (input/print)	Meets the no-GUI execution requirement
Version control	Git & GitHub	Required for submission and change tracking

5. System Design

5.1 Data Model

The application keeps two lists inside a single JSON file, `data.json`: one for routes and one for bookings. Each route record stores a route ID, source, destination, total seats, the list of seats already booked, and the fare per seat. Each booking record stores a unique booking ID, the passenger's name, the route ID it belongs to, the list of seats assigned, and the total fare paid. Keeping bookings separate from routes, but linked by route ID, avoids duplicating route information for every booking.

5.2 Program Modules (Functions)

Function	Responsibility
load_data() / save_data()	Read and write data.json so records survive between runs
display_routes()	Print all routes with live seat availability and fare
book_seat()	Validate input, assign free seat numbers, create a new booking
cancel_booking()	Look up a booking by ID, free its seats, remove the record
search_passenger()	Find and display every booking under a given passenger name
calculate_fare()	Compute cost for a route and seat count without booking
main() / show_menu()	Drive the command-line menu loop

6. Features Implemented

- **Display Routes** – lists every route with live count of remaining seats and the fare per seat.
- **Book a Seat** – lets a passenger reserve one or more seats on a chosen route; seat numbers and a booking ID are generated automatically.
- **Cancel a Booking** – frees the reserved seats and removes the booking using its booking ID.
- **Search Passenger** – retrieves every booking made under a given passenger name (case-insensitive).
- **Calculate Fare** – previews the total cost for a route and seat count before committing to a booking.
- **Persistent Storage** – all routes and bookings are saved to data.json, so nothing is lost when the program closes.

7. Installation and Execution Steps

1. Ensure Python 3.7 or above is installed (check with `python3 --version`).
2. Clone the repository: `git clone https://github.com/your-username/your-repo-name.git`
3. Move into the project folder: `cd your-repo-name`
4. Run the program: `python3 bus_booking.py`
5. Follow the on-screen numbered menu to display routes, book seats, cancel bookings, search a passenger, or calculate fare.

No external libraries are required — the project uses only Python's standard library (json, os, random), so there is nothing extra to install.

8. Sample Output

Below is a sample run of the program showing the main menu and a route listing.

```
===== BUS BOOKING SYSTEM =====
1. Display Routes
2. Book a Seat
3. Cancel a Booking
4. Search Passenger
5. Calculate Fare
6. Exit
=====
Enter your choice (1-6): 1

----- AVAILABLE BUS ROUTES -----
ID    From      To      Seats Left  Fare
-----
1     Bhopal     Indore    40         350
2     Indore     Ujjain    30         150
3     Bhopal     Gwalior   35         500
-----
```

(Replace this with an actual screenshot of your own terminal output before submitting.)

9. Testing

The program was tested manually by running through each menu option: booking multiple seats on a route until it became full, attempting to book more seats than were available, cancelling a booking and confirming the seats were released back to the route, searching for a passenger with no bookings, and calculating fares for different seat counts. Invalid inputs such as text entered instead of numbers were also tested to confirm the program shows an error instead of crashing.

10. Challenges Faced

[Write 2–4 sentences here, in your own words, about something you personally found tricky — for example, handling seat numbering when seats are cancelled and re-booked, or deciding how to structure the JSON file. Personalizing this section is important since it reflects your own experience working on the project.]

11. Learning Outcomes

[Write a short paragraph here on what you learned — for example, working with Python dictionaries and lists, reading/writing JSON files, structuring a menu-driven program, or handling user input safely with try/except.]

12. Future Enhancements

- Add an admin mode to add, edit, or remove routes.
- Support multiple trip timings/dates per route.
- Add a simple seat-map visualization (rows and columns of seats).
- Move from JSON storage to a proper database (e.g. SQLite) for larger datasets.
- Add automated unit tests using Python's unittest module.

13. Conclusion

This project demonstrates a complete, working command-line booking system built using core Python concepts covered in the Python Essentials course. It fulfills the key needs of a small booking operation — viewing routes, booking and cancelling seats, searching records, and calculating fares — while keeping the codebase simple enough for a beginner to fully understand, extend, and maintain.